18-449/649 Lab 5: Time Synchronization and Consensus

Misho Alexandrov (mvalexan), Ore Abolade (oabolade), T'sairus Beasley (tbeasley)

Repository: github.com/o-abolade/18-649-Tabletop-Drive-by-Wire-Car · code as of commit `5c54aca` (2026-11-13), in `Lab5_Time_Sync/`. The design is in `design.md`, and every measurement with its method is in `Progress_doc_lab5.md`.

System: N1 cockpit (Raspberry Pi 4), N2 steering and N3 drivetrain (Nucleo-F401RE, Zephyr) on two 500 kbit/s CAN buses. All three nodes run the same Raft core (`common/raft/`, plain C, also covered by 38 host tests). The Raft leader is the time master.

1. Time drift adjustment: detecting it and applying it

What is measured. The leader sends `REF` every 20 ms (the Raft heartbeat). Each node timestamps the *end of the frame on the wire*:

- STM32s: an ISR on the MCP2515 INT line records the cycle counter when INT falls, at transmit-complete on the leader and at receive on the followers;
- Pi: kernel `SO_TIMESTAMPNS` on received frames and on the echo of each sent frame, converted to `CLOCK_MONOTONIC_RAW` so NTP adjustments don't look like drift.

`REF` k carries the leader's send time of `REF` k−1 *on the same bus* (follow-up timestamp). A follower pairs it with its own receive stamp of k−1, which gives one sample (L, F) per REF.

Offset (handout 6.1). Each accepted sample re-anchors the clock: `global(F) = L + network_delay`. The delay is −2 µs: the receiving MCP2515 flags a frame one bit before the sender flags transmit-complete (measured: the follower's PPS led by 1.1–2.0 µs before this calibration). A `REF` that only arrived through a relay gets +400 µs.

Drift ratio (handout 6.2). `(L2 − L1) / (F2 − F1)` over pairs at least 4 s apart, low-pass filtered (weight 1/4). Over a 1 s baseline, the Pi's ±40 µs timestamp jitter read as ±40–80 ppm, so a longer baseline is needed.

Applying it. `global_gettime() = anchor_global + Δlocal · (1 + drift)`, with Δlocal the local time since the last sample. The STM32s also set the Zephyr clock (`sys_clock_settime`) on each sample; the Pi keeps its own offset. The drift term matters exactly when `REF`s stop: during an election or relay start-up each node free-runs on its own crystal, corrected by its ratio.

Deciding when a sample is trusted:

- a stamp older than 200 µs when the driver hands over the frame may belong to another frame, so it is not used;
- a sample more than 150 µs off is an outlier; ten in a row means the master time really moved (a different leader), and the clock steps;
- samples come from one bus at a time, since the two buses differ by a few µs and mixing them gave a −10 µs saw-tooth;
- SYNC is valid while a sample arrived in the last 60 ms;
- a new leader freezes its mapping and continues its own global time, so the followers see no step at a handover.

Measured on hardware (scope, consoles)	Result
PPS spread N2 vs N3 (STM32 leader)	< 1 µs
Hazards FL vs RL / FL vs HZ (Pi leader)	0.5 µs / 46 µs
Hazards FL vs RL at 4 Hz, after the Pi failed	0.74 µs, no visible step
Turn signals FL vs RL / FR vs RR	0 s / 1 µs
Holdover 83 s without `REF`, then resync	6 µs (uncorrected: about 360 µs)
Drift N3 vs N2 / Pi vs STM32s	−5.4 ppm / about 95 ppm
Pi following through a relay	worst 90–150 µs per second, no steps

2. The Raft state machine

State. The network graph from the handout:

- node X up/down (3 bits);
- the link of node X to bus Y up/down (6 bits);
- cockpit OK/ERROR (1 bit, failure mode).

It packs into 16 bits, so a snapshot fits in one CAN frame. The initial state (index 0) has everything up and the cockpit in ERROR: the system powers up in error (Lab 1 R3), so the first `COCKPIT_OK` is a log entry.

Log entries are (term, index, event), with an 8-bit term, a 16-bit index and a 1-byte event. There are 20 event codes, each mapped to one handout mutator:

Event	Mutator	Effect
`CANLINK_X_Y_UP` / `_DOWN`	`addLink(X,Y)` / `dropLink(X,Y)`	mark one link
`NODE_X_UP`	`addNode(X)`	node back, *with no links* until its `addLink`s
`NODE_X_DOWN`	`dropNode(X)`	node and its links removed
`COCKPIT_OK` / `_ERROR`	—	leave / enter failure mode (4 Hz hazards)

`getNetwork()` returns a copy of the graph.

Commit and apply. The leader appends a client report only if it changes the *projected* state (committed plus pending entries). Three nodes reporting the same failure therefore produce one entry. An entry commits once a majority (2 of 3) holds it, and every node applies committed entries in index order. Entries that reach a node already committed (catch-up after a reboot) are marked `replayed`, which is how the Pi shows held events exactly once and in order when it rejoins (checkoff 7).

Log storage. `log_size` is 30000 logical entries. The STM32s keep the 256 newest in a RAM ring plus a snapshot of the graph as of the oldest one (a 30000-entry log would not fit in 96 KB). A node that missed compacted entries is caught up with one `SNAPSHOT` frame.

Raft mechanics around it:

- predefined leader: every node boots in term 1 following the cockpit, with no election (checkoff 1–2);
- pre-vote: an isolated node never raises the term (checkoff 10);
- check-quorum: an isolated leader steps down;
- term guard: votes and acknowledgements more than 32 terms ahead are dropped, and every Raft frame carries a CRC-4 in byte 0;
- LEADER=random: the election timer starts once all three nodes are heard, so the seed, not the boot order, picks the first leader (checkoff 8).

Bugs the hardware found, and what fixed them:

- **A rebooted follower was never caught up.** Our logs are in RAM, unlike Raft's persistent logs. The fix: a follower reporting a shorter log than its match index resets that match, and followers are served in turn so a stuck one cannot starve the other.
- **Garbled frames moved the cluster to term 255.** The cause was a loose SPI wire on the Pi; the CRC and the term guard now stop such frames.

In every hardware run, all three nodes' logs were identical (same entry at the same index), across link faults, power cycles, isolation and leader changes.

3. The Raft client and CAN routing during multi-link failures

Client (`netstate_observe`, every 10 ms on every node). Each node watches the Lab 3 heartbeats per bus and reports only what it can tell apart:

- **its own links:** link Y is up if it hears *anyone* on bus Y within 60 ms (a cut jumper cuts both directions);
- **other nodes:** node X is down if it is unheard on both buses for 200 ms. For the cockpit this also means `COCKPIT_ERROR`;
- **nothing about others while it hears nobody.** An isolated node cannot tell whether it failed or they did.

Reports go to the leader as `EVENT` frames. A follower answers with a redirect to the leader, and the client retries every 50 ms until it gets `OK` or `NO_CHANGE`. Queued reports that are no longer observed are withdrawn, so a report queued while a node was cut off is not committed after it reconnects.

Routing from `getNetwork()`. Each node derives its routing table from the committed graph:

- X's frames arriving on bus b are relayed to the other bus when some live node is on the other bus but not on b, and X is not on the other bus;
- the relay is the lowest-numbered relay-capable node on both buses, so every node agrees on it and exactly one copy is forwarded;
- the Pi never relays (no worst-case timing bound on Linux).

This replaces Lab 3's always-on relay.

The multi-link edge case (handout 5.2). After a second link fails, the committed graph is out of date until that failure is reported and committed, and the report itself may need the relay. Two mechanisms close the gap:

1. **Heartbeats carry link health.** Each heartbeat holds the sender's own view of its links. Each node merges these into a local copy of the graph and relays if *either* the committed or the local graph says so, so the relay starts about 80–100 ms after the fault, before anything commits.
2. **Node-down needs 200 ms, links only 60 ms.** At 60 ms, a node that could hear another only through the not-yet-started relay reported it down; the cockpit was committed down while it was fine (seen on hardware). With 200 ms, the relay is running first.

`REF` is the one frame with different data per bus (each bus's send time), so it is relayed only when the leader's own heartbeat says it is off the target bus. A relayed copy can then never collide with the leader's copy (same ID, different data, the Lab 3 bit-error problem).

Checkoff 5 on hardware (Pi off B, N3 off A, Pi leading). N2 is the only node on both buses and becomes the relay. N3 missed the leader for 8 ms while the relay started; pre-vote kept the term unchanged. N3's `CANLINK_drivetrain_A_DOWN` reached the leader through N2. Exactly two entries were committed (`CANLINK_drivetrain_A_DOWN`, `CANLINK_cockpit_B_DOWN`), at the same index on all three nodes.

With an STM32 leader (N2; Pi off B, N2 off A), N3 relayed about 194 frames/s and the Pi stayed synced through the relayed `REF` (worst 90–150 µs per second).

Not covered: N2 and N3 on opposite buses. Only the Pi could bridge them, and it does not relay (open question to the course staff).